

01_docker-compose.yml

#docker

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ docker/  
    └─ compose.yml
```

El archivo `compose.yml` se encuentra dentro de la carpeta `docker/` del proyecto. Su función es definir los contenedores necesarios para levantar el laboratorio SIEM MVP.

Este archivo no contiene lógica de programación del SIEM como tal, sino la configuración de infraestructura necesaria para ejecutar el entorno:

```
PostgreSQL → Base de datos  
Adminer    → Interfaz web para consultar PostgreSQL  
FastAPI     → API backend del laboratorio
```

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,220p' docker/compose.yml
```

El comando `sed -n '1,220p' docker/compose.yml` muestra desde la línea 1 hasta la línea 220 del archivo `compose.yml`.

Desglose del comando:

`sed`

Ejecuta el programa `sed`, utilizado para leer, filtrar o transformar texto.

`-n`

Evita que `sed` imprima todo el archivo automáticamente.

`'1,220p'`

Indica que se impriman las líneas de la 1 a la 220.

`docker/compose.yml`

Es la ruta del archivo que se quiere leer.

3 Código completo del archivo

```
services:  
  db:  
    image: postgres:16  
    container_name: siem-db  
    environment:  
      POSTGRES_DB: ${POSTGRES_DB:?set in docker/.env}  
      POSTGRES_USER: ${POSTGRES_USER:?set in docker/.env}  
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set in docker/.env}  
      POSTGRES_HOST_AUTH_METHOD: scram-sha-256  
    volumes:  
      - siem_db:/var/lib/postgresql/data  
  networks:
```

```

- siem-net
restart: unless-stopped
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
  interval: 5s
  timeout: 3s
  retries: 10

adminer:
  image: adminer:4
  container_name: siem-adminer
  ports:
    - "127.0.0.1:${ADMINER_PORT:-8080}:8080"
  networks:
    - siem-net
  restart: unless-stopped
  depends_on:
    db:
      condition: service_healthy

api:
  build:
    context: ../backend
    dockerfile: Dockerfile
  container_name: siem-api
  working_dir: /app
  volumes:
    - ../backend:/app
  environment:
    DATABASE_URL: ${DATABASE_URL:?set in docker/.env}
    APP_VERSION: ${APP_VERSION:-0.1.0}
    GIT_SHA: ${GIT_SHA:-unknown}
    BUILD_TIME: ${BUILD_TIME:-unknown}
  networks:
    - siem-net
  ports:
    - "127.0.0.1:${API_PORT:-8000}:8000"
  depends_on:
    db:
      condition: service_healthy
  command: uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

volumes:
  siem_db:

networks:
  siem-net:
    name: siem-net

```

4 Función general del archivo

El archivo `compose.yml` define cómo se levanta el laboratorio SIEM mediante Docker Compose.

Docker Compose permite declarar varios servicios en un único archivo. En este caso se definen tres servicios:

```

db      → Base de datos PostgreSQL
adminer → Herramienta web para consultar la base de datos
api     → Backend FastAPI del laboratorio

```

También se definen dos elementos comunes:

```

volumes → almacenamiento persistente para PostgreSQL
networks → red interna para comunicar los contenedores

```

El objetivo de este archivo es que el laboratorio pueda arrancarse con un único comando, normalmente:

```
docker compose --env-file docker/.env -f docker/compose.yml up -d
```

A nivel de arquitectura, este archivo permite que la API, la base de datos y Adminer trabajen dentro de un mismo entorno controlado.

5 Estructura general del archivo

El archivo tiene cuatro bloques principales:

```
services:
```

Define los contenedores que forman parte del laboratorio.

```
volumes:
```

Define los volúmenes persistentes.

```
networks:
```

Define las redes internas de Docker.

Dentro de `services`, aparecen tres servicios:

```
db:
adminer:
api:
```

Cada servicio representa un contenedor diferente.

La estructura general puede entenderse así:

```
compose.yml
├── services
│   ├── db
│   ├── adminer
│   └── api
├── volumes
│   └── siem_db
└── networks
    └── siem-net
```

6 Análisis línea por línea

Bloque `services`

```
services:
```

La palabra `services` es una clave principal de Docker Compose.

Indica que a partir de aquí se van a definir los servicios o contenedores que forman parte de la aplicación.

En Docker Compose, un servicio equivale normalmente a un contenedor configurado. En este proyecto hay tres servicios:

```
db
adminer
api
```

Cada uno tendrá su propia imagen, nombre, variables, red, puertos y comportamiento.

7 Servicio db

Definición del servicio

```
db:
```

db es el nombre del servicio de base de datos.

La indentación de dos espacios indica que db está dentro de services .

Este nombre es importante porque Docker Compose permite que otros contenedores se conecten a este servicio usando el nombre db como host.

Por ejemplo, la API puede conectarse a PostgreSQL usando una URL similar a:

```
postgresql+psycpg://usuario:password@db:5432/base_de_datos
```

Aquí db no es una IP, sino el nombre del servicio dentro de la red de Docker.

Imagen de PostgreSQL

```
image: postgres:16
```

La clave image indica qué imagen Docker se utilizará para crear el contenedor.

```
postgres:16
```

Significa:

```
postgres → nombre de la imagen
16        → versión o tag de la imagen
```

En este caso se utiliza PostgreSQL versión 16.

Esto evita instalar PostgreSQL manualmente en el sistema operativo. Docker descarga la imagen y ejecuta la base de datos dentro de un contenedor.

Nombre del contenedor

```
container_name: siem-db
```

container_name define el nombre real que tendrá el contenedor cuando esté ejecutándose.

En este caso:

```
siem-db
```

Esto permite identificarlo fácilmente con comandos como:

```
docker ps
docker logs siem-db
docker exec -it siem-db bash
```

Sin container_name , Docker Compose generaría un nombre automático basado en la carpeta del proyecto y el nombre del servicio.

Variables de entorno

```
environment:
```

La clave `environment` define variables de entorno dentro del contenedor.

Estas variables son utilizadas por la imagen oficial de PostgreSQL para inicializar la base de datos.

Variable `POSTGRES_DB`

```
POSTGRES_DB: ${POSTGRES_DB:?set in docker/.env}
```

`POSTGRES_DB` indica el nombre de la base de datos que PostgreSQL creará al iniciar por primera vez.

La parte derecha usa sustitución de variables de Docker Compose:

```
${POSTGRES_DB:?set in docker/.env}
```

Desglose:

```
${...}      → sintaxis para leer una variable de entorno
POSTGRES_DB → nombre de la variable
:?          → indica que la variable es obligatoria
set in docker/.env → mensaje de error si no está definida
```

Esto significa:

```
Lee la variable POSTGRES_DB desde el entorno o desde docker/.env.
Si no existe, detén el arranque y muestra el mensaje "set in docker/.env".
```

Es una buena práctica porque evita arrancar la base de datos sin nombre definido.

Variable `POSTGRES_USER`

```
POSTGRES_USER: ${POSTGRES_USER:?set in docker/.env}
```

`POSTGRES_USER` define el usuario principal de PostgreSQL.

Sigue la misma lógica que la variable anterior:

```
Debe estar definida obligatoriamente.
Si no existe, Docker Compose detiene el arranque.
```

Este usuario será utilizado posteriormente por la API para conectarse a la base de datos.

Variable `POSTGRES_PASSWORD`

```
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set in docker/.env}
```

`POSTGRES_PASSWORD` define la contraseña del usuario de PostgreSQL.

También es obligatoria.

Esto evita dejar una base de datos sin contraseña o con una configuración incompleta.

Desde el punto de vista de seguridad, las credenciales no se escriben directamente en el `compose.yml`, sino en un archivo `.env`.

Método de autenticación

```
POSTGRES_HOST_AUTH_METHOD: scram-sha-256
```

Esta variable configura el método de autenticación usado por PostgreSQL para conexiones desde otros hosts.

```
scram-sha-256
```

es un método de autenticación más seguro que métodos antiguos como `md5`.

En el contexto del proyecto, esto refuerza la configuración de la base de datos aunque sea un entorno de laboratorio.

Volumen de PostgreSQL

```
volumes:
```

La clave `volumes` define montajes de almacenamiento para el contenedor.

En este caso se usa para conservar los datos de PostgreSQL.

```
- siem_db:/var/lib/postgresql/data
```

Esta línea monta el volumen llamado `siem_db` en la ruta interna:

```
/var/lib/postgresql/data
```

Esa ruta es donde PostgreSQL guarda físicamente sus datos dentro del contenedor.

Desglose:

<code>siem_db</code>	→ nombre del volumen Docker
<code>:</code>	→ separador entre origen y destino
<code>/var/lib/postgresql/data</code>	→ ruta interna del contenedor

Esto significa:

Guarda los datos de PostgreSQL en el volumen `siem_db`.
Dentro del contenedor, esos datos estarán en `/var/lib/postgresql/data`.

Gracias a esto, si se elimina o recrea el contenedor `siem-db`, los datos no se pierden mientras el volumen siga existiendo.

Red del servicio `db`

```
networks:
```

La clave `networks` indica a qué redes Docker se conectará el servicio.

```
- siem-net
```

El servicio `db` se conecta a la red llamada `siem-net`.

Esto permite que otros servicios conectados a la misma red, como `api` o `adminer`, puedan comunicarse con la base de datos.

Política de reinicio

```
restart: unless-stopped
```

La clave `restart` define qué debe hacer Docker si el contenedor se detiene.

```
unless-stopped
```

significa:

Reinicia el contenedor automáticamente si falla o si Docker se reinicia, excepto si el usuario lo ha parado manualmente.

Esto aporta estabilidad al laboratorio, porque la base de datos intenta mantenerse disponible.

Healthcheck de PostgreSQL

```
healthcheck:
```

La clave `healthcheck` define una comprobación de salud del contenedor.

No basta con que el contenedor esté encendido. PostgreSQL puede tardar unos segundos en aceptar conexiones. El `healthcheck` permite saber cuándo está realmente listo.

```
test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
```

La clave `test` define el comando que Docker ejecutará para comprobar el estado del servicio.

Se usa formato de lista:

```
["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
```

Desglose:

CMD-SHELL → ejecuta el comando mediante una shell
pg_isready → utilidad de PostgreSQL para comprobar disponibilidad
-U \${POSTGRES_USER} → usuario con el que se comprueba la conexión
-d \${POSTGRES_DB} → base de datos objetivo

Este comando pregunta a PostgreSQL:

```
¿Estás listo para aceptar conexiones con este usuario y esta base de datos?
```

Si la respuesta es correcta, el contenedor pasa a estado `healthy`.

```
interval: 5s
```

`interval` indica cada cuánto se ejecuta el `healthcheck`.

En este caso:

```
cada 5 segundos
```

```
timeout: 3s
```

`timeout` indica cuánto tiempo espera Docker la respuesta del comando antes de considerarlo fallido.

En este caso:

```
3 segundos
```

```
retries: 10
```

`retries` indica cuántos intentos fallidos se permiten antes de marcar el contenedor como no saludable.

En este caso:

```
10 intentos
```

La combinación completa significa:

```
Docker comprobará cada 5 segundos si PostgreSQL está listo.  
Cada comprobación tendrá un máximo de 3 segundos para responder.  
Si falla 10 veces, el servicio se considerará no saludable.
```

8 Servicio adminer

Definición del servicio

```
adminer:
```

`adminer` es el nombre del servicio encargado de levantar Adminer.

Adminer es una interfaz web para gestionar bases de datos desde el navegador.

En este laboratorio se utiliza para revisar manualmente la base de datos PostgreSQL: tablas, eventos, reglas y alertas.

Imagen de Adminer

```
image: adminer:4
```

Esta línea indica que el contenedor se creará usando la imagen oficial de Adminer, versión 4.

Desglose:

```
adminer → nombre de la imagen  
4       → versión o tag
```

Nombre del contenedor

```
container_name: siem-adminer
```

El contenedor recibirá el nombre:

```
siem-adminer
```

Esto facilita identificarlo con comandos como:

```
docker ps  
docker logs siem-adminer
```

Publicación de puertos

```
ports:
```

La clave `ports` permite exponer un puerto del contenedor hacia la máquina anfitriona.


```
- "127.0.0.1:${ADMINER_PORT:-8080}:8080"
```

Esta línea tiene tres partes principales:

127.0.0.1	→ dirección de escucha en el host
\${ADMINER_PORT:-8080}	→ puerto externo en la máquina anfitriona
8080	→ puerto interno del contenedor

La estructura general es:

```
HOST:PUERTO_EXTERNO:PUERTO_INTERNO
```

Desglose completo:

```
127.0.0.1
```

Limita el acceso a la propia máquina local. Adminer no queda expuesto a toda la red.

```
${ADMINER_PORT:-8080}
```

Lee la variable `ADMINER_PORT` . Si no existe, usa `8080` por defecto.

La sintaxis `: -` significa:

```
usa este valor por defecto si la variable no está definida
```

```
8080
```

Es el puerto interno donde Adminer escucha dentro del contenedor.

Por tanto, si `ADMINER_PORT` no se define, Adminer estará disponible en:

```
http://localhost:8080
```

Red de Adminer

```
networks:  
- siem-net
```

Adminer se conecta a la red `siem-net` .

Esto le permite comunicarse con PostgreSQL usando el nombre del servicio:

```
db
```

Al conectarse desde Adminer, el servidor de base de datos será:

```
db
```

No hace falta usar una IP interna.

Política de reinicio

```
restart: unless-stopped
```

Adminer se reiniciará automáticamente salvo que se haya detenido manualmente.

Dependencia de PostgreSQL

```
depends_on:
```

La clave `depends_on` define dependencias entre servicios.

```
db:
  condition: service_healthy
```

Esto significa que Adminer depende del servicio `db`.

La condición:

```
condition: service_healthy
```

indica que Adminer no debe arrancar hasta que PostgreSQL esté marcado como saludable por su `healthcheck`.

Esto evita que Adminer arranque antes de que la base de datos esté lista.

9 Servicio `api`

Definición del servicio

```
api:
```

`api` es el servicio que ejecuta el backend FastAPI del laboratorio SIEM.

Este contenedor será el núcleo de la aplicación, ya que expone los endpoints para trabajar con eventos, reglas, alertas, métricas e información del sistema.

Construcción de la imagen

```
build:
```

La clave `build` indica que Docker no usará directamente una imagen externa, sino que construirá una imagen propia.

```
context: ../backend
```

`context` indica la carpeta que Docker usará como contexto de construcción.

En este caso:

```
../backend
```

Como el archivo `compose.yml` está dentro de `docker/`, `../backend` significa:

```
sube un nivel desde docker/ y entra en backend/
```

Visualmente:

```
siem-lab/
├── docker/
│   └── compose.yml
└── backend/
    └── Dockerfile
```

Desde `docker/compose.yml`, la ruta `../backend` apunta a `backend/`.

```
dockerfile: Dockerfile
```

Indica qué archivo Dockerfile debe utilizarse dentro del contexto definido.

Como el contexto es `../backend`, Docker buscará:

```
backend/Dockerfile
```

Nombre del contenedor API

```
container_name: siem-api
```

El contenedor de la API se llamará:

```
siem-api
```

Esto permite identificarlo fácilmente:

```
docker logs siem-api
docker exec -it siem-api bash
```

Directorio de trabajo

```
working_dir: /app
```

`working_dir` define el directorio de trabajo dentro del contenedor.

Esto significa que los comandos ejecutados dentro del contenedor se ejecutarán desde:

```
/app
```

En este proyecto, `/app` será la carpeta donde se encuentra el código del backend.

Montaje del código fuente

```
volumes:
```

La clave `volumes` también puede usarse para montar carpetas locales dentro del contenedor.

```
- ../backend:/app
```

Esta línea monta la carpeta local `backend/` en la ruta `/app` del contenedor.

Desglose:

```
../backend → carpeta del host
:          → separador entre host y contenedor
/app       → ruta dentro del contenedor
```

Esto permite que el contenedor use el código fuente real del proyecto.

Ventaja principal:

```
Si se modifica el código en el host, el cambio aparece dentro del contenedor.
```

Combinado con `--reload`, permite desarrollo en caliente.

Variables de entorno de la API

```
environment:
```

Define variables de entorno disponibles dentro del contenedor `siem-api`.

Variable `DATABASE_URL`

```
DATABASE_URL: ${DATABASE_URL:?set in docker/.env}
```

`DATABASE_URL` contiene la cadena de conexión que usa la API para conectarse a PostgreSQL.

La sintaxis:

```
${DATABASE_URL:?set in docker/.env}
```

significa que la variable es obligatoria.

Si no está definida, Docker Compose detiene el arranque.

Una `DATABASE_URL` típica puede tener esta estructura conceptual:

```
postgresql+psycpg://usuario:password@db:5432/base_de_datos
```

Desglose:

postgresql+psycpg	→ dialecto y driver
usuario	→ usuario de PostgreSQL
password	→ contraseña
db	→ host, en este caso el nombre del servicio Docker
5432	→ puerto interno de PostgreSQL
base_de_datos	→ nombre de la base de datos

Variable `APP_VERSION`

```
APP_VERSION: ${APP_VERSION:-0.1.0}
```

`APP_VERSION` define la versión de la aplicación.

La sintaxis:

```
${APP_VERSION:-0.1.0}
```

significa:

```
usa APP_VERSION si existe;  
si no existe, usa 0.1.0 como valor por defecto.
```

Esta variable puede utilizarse después en endpoints informativos, por ejemplo `/info`.

Variable `GIT_SHA`

```
GIT_SHA: ${GIT_SHA:-unknown}
```

`GIT_SHA` permite guardar el identificador del commit de Git con el que se construyó o ejecutó la aplicación.

Si la variable no existe, toma el valor:

```
unknown
```

Esto es útil para trazabilidad.

Variable `BUILD_TIME`

```
BUILD_TIME: ${BUILD_TIME:-unknown}
```

`BUILD_TIME` permite indicar la fecha u hora de construcción de la aplicación.

Si no está definida, usa:

```
unknown
```

También sirve para trazabilidad y diagnóstico.

Red de la API

```
networks:
  - siem-net
```

El contenedor `api` se conecta a la red interna `siem-net`.

Gracias a esto puede comunicarse con PostgreSQL usando el nombre del servicio:

```
db
```

Puerto de la API

```
ports:
```

Permite exponer el puerto interno de la API hacia la máquina anfitriona.

```
- "127.0.0.1:${API_PORT:-8000}:8000"
```

Desglose:

<code>127.0.0.1</code>	→ solo accesible desde la máquina local
<code>\${API_PORT:-8000}</code>	→ puerto externo configurable, por defecto 8000
<code>8000</code>	→ puerto interno del contenedor

La API se podrá abrir normalmente en:

```
http://localhost:8000
```

Y la documentación Swagger de FastAPI en:

```
http://localhost:8000/docs
```

La dirección `127.0.0.1` evita que la API quede expuesta directamente a otros equipos de la red local.

Dependencia de PostgreSQL

```
depends_on:
  db:
    condition: service_healthy
```

La API depende de la base de datos.

Gracias a esta configuración, Docker Compose espera a que `db` esté en estado `healthy` antes de arrancar `api`.

Esto evita errores típicos como:

```
connection refused
database unavailable
could not connect to server
```

Comando de arranque de la API

```
command: uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

La clave `command` sobrescribe el comando por defecto definido en el Dockerfile.

Este comando arranca la aplicación FastAPI mediante Uvicorn.

Desglose:

```
uvicorn
```

Ejecuta el servidor ASGI.

FastAPI no se ejecuta directamente por sí solo, necesita un servidor compatible ASGI. En este caso se usa Uvicorn.

```
app.main:app
```

Indica dónde está la aplicación FastAPI.

Se interpreta así:

```
app      → paquete/carpeta app/
main     → archivo main.py
app      → variable app definida dentro de main.py
```

Por tanto, Uvicorn buscará:

```
backend/app/main.py
```

y dentro de ese archivo una variable llamada:

```
app
```

```
--reload
```

Activa la recarga automática. Si se modifica el código, Uvicorn reinicia la aplicación.

Es útil en desarrollo, pero no sería lo normal en producción.

```
--host 0.0.0.0
```

Hace que Uvicorn escuche en todas las interfaces de red dentro del contenedor.

Esto es necesario para que Docker pueda mapear el puerto hacia el host.

```
--port 8000
```

Indica que la API escucha dentro del contenedor en el puerto `8000`.

```
volumes:
```

Esta clave principal define volúmenes gestionados por Docker.

Un volumen permite guardar datos fuera del ciclo de vida del contenedor.

```
siem_db:
```

Define un volumen llamado `siem_db`.

Este volumen se usa anteriormente en el servicio `db`:

```
- siem_db:/var/lib/postgresql/data
```

Gracias a este volumen, los datos de PostgreSQL permanecen aunque el contenedor se destruya y se vuelva a crear.

1 1 Bloque networks

```
networks:
```

Esta clave principal define redes personalizadas de Docker.

```
siem-net:
```

Define una red llamada `siem-net`.

Los servicios conectados a esta red pueden comunicarse entre sí.

```
name: siem-net
```

Define explícitamente el nombre real de la red Docker.

Sin esta línea, Docker Compose podría crear un nombre automático combinando el nombre del proyecto y el nombre de la red.

Con esta línea, la red se llamará exactamente:

```
siem-net
```

Esto facilita identificarla con:

```
docker network ls
docker network inspect siem-net
```

1 2 Flujo de arranque del laboratorio

El archivo `compose.yml` permite este flujo:

1. Docker Compose lee el archivo `docker/compose.yml`.
2. Docker Compose carga las variables desde `docker/.env`.
3. Se crea la red interna `siem-net` si no existe.
4. Se crea el volumen `siem_db` si no existe.
5. Arranca el contenedor `siem-db` con PostgreSQL 16.
6. PostgreSQL inicializa la base de datos usando `POSTGRES_DB`, `POSTGRES_USER` y `POSTGRES_PASSWORD`.
7. El healthcheck ejecuta `pg_isready`.
8. Cuando PostgreSQL está listo, el servicio `db` pasa a `healthy`.
9. Adminer puede arrancar porque depende de `db` `healthy`.

10. La API puede arrancar porque también depende de db healthy.
11. La API ejecuta Uvicorn con `app.main:app`.
12. Uvicorn carga `backend/app/main.py`.
13. FastAPI queda disponible en `http://localhost:8000`.
14. Adminer queda disponible en `http://localhost:8080`.

1 3 Relación con el flujo de datos del SIEM

Este archivo no procesa eventos ni genera alertas, pero prepara el entorno donde ese flujo puede ocurrir.

La relación con el flujo de datos es:

```
compose.yml
  ↓
levanta PostgreSQL + API
  ↓
la API recibe eventos
  ↓
la API guarda eventos en PostgreSQL
  ↓
la API consulta reglas
  ↓
la API genera alertas
  ↓
Adminer permite revisar manualmente los datos
```

Sin este archivo, habría que instalar y configurar manualmente cada componente.

1 4 Errores típicos relacionados

♦ Falta una variable obligatoria

Si falta una variable como `POSTGRES_DB`, aparecerá un error relacionado con:

```
set in docker/.env
```

Esto ocurre por la sintaxis:

```
${POSTGRES_DB:?set in docker/.env}
```

Solución: revisar el archivo `docker/.env`.

♦ La API no conecta con PostgreSQL

Puede deberse a una `DATABASE_URL` incorrecta.

Dentro de Docker Compose, el host debe ser normalmente:

```
db
```

No `localhost`.

`localhost` dentro del contenedor `api` apuntaría al propio contenedor de la API, no al contenedor de PostgreSQL.

♦ Adminer no abre

Revisar si el puerto configurado es el correcto:

```
${ADMINER_PORT:-8080}
```


Si no se ha definido `ADMINER_PORT`, se usa `8080`.

♦ La API no abre

Revisar si el puerto configurado es:

```
${API_PORT:-8000}
```

Si no se ha definido `API_PORT`, se usa `8000`.

♦ Cambios en código no se reflejan

El montaje:

```
- ../backend:/app
```

junto con:

```
--reload
```

debería reflejar cambios automáticamente.

Si no ocurre, revisar que el contenedor esté usando el volumen correcto y que Uvicorn esté arrancado con `--reload`.

15 Comandos útiles relacionados

Levantar el laboratorio:

```
docker compose --env-file docker/.env -f docker/compose.yml up -d
```

Ver contenedores activos:

```
docker ps
```

Ver logs de la API:

```
docker logs siem-api
```

Ver logs de PostgreSQL:

```
docker logs siem-db
```

Ver logs de Adminer:

```
docker logs siem-adminer
```

Entrar en el contenedor de la API:

```
docker exec -it siem-api bash
```

Entrar en PostgreSQL:

```
docker exec -it siem-db psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"
```

Ver redes Docker:

```
docker network ls
```

Inspeccionar la red del laboratorio:

```
docker network inspect siem-net
```

Ver volúmenes Docker:

```
docker volume ls
```

16 Cómo explicarlo en la presentación

El archivo `docker/compose.yml` define la infraestructura principal del laboratorio SIEM MVP. En lugar de instalar manualmente la base de datos, la herramienta de administración y la API, se declaran como servicios independientes dentro de Docker Compose.

El servicio `db` levanta PostgreSQL 16 y conserva los datos mediante un volumen persistente. El servicio `adminer` proporciona una interfaz web para inspeccionar la base de datos. El servicio `api` construye y ejecuta el backend FastAPI, conectándose a PostgreSQL mediante la variable `DATABASE_URL`.

Los tres servicios comparten una red interna llamada `siem-net`, lo que permite que la API y Adminer se comuniquen con la base de datos usando el nombre del servicio `db`. Además, los puertos se publican en `127.0.0.1`, limitando el acceso a la máquina local durante el desarrollo.

Esta configuración hace que el laboratorio sea reproducible, portable y fácil de levantar en distintos entornos.